

Week2 Report

报告人：张筱涵

一、实验思路

本周的工作是基于 Black-Scholes-Merton 框架复现 Chooser 期权定价模型。Chooser 期权的定价公式可以通过 Put-Call Parity 推导得出：

$$V_0 = C(S, K, T, r, \sigma) + P(S, K e^{-r(T-t_c)}, t_c, r, \sigma)$$

即一个到期日为 T 的欧式看涨期权，加上一个行权价经过折现、到期日为选择日 t_c 的欧式看跌期权。

本周采用两套参数分别进行验证：第一套沿用论文原始参数 ($K=150$, $T=1$ 年, $\sigma=20\%$, $r=5\%$)，用于验证模型公式的正确性；第二套采用平值期权参数 ($K=S$, 行权价等于当日市场价格)，用于对真实市场数据进行定价，确保 Call 和 Put 两个部分均有实质性价值，更真实地反映 Chooser 期权的定价特征。

在 BSM 定价结果的基础上，评估模型在真实市场环境下的表现与局限性。评估有三个方面：一是波动率稳定性检验，通过计算变异系数 (CV) 量化 BSM 恒定波动率假设与真实市场的偏差；二是制度依赖性分析，结合第一周构建的 VIX 制度分类，对比高、中、低波动率制度下的定价差异；三是极端市场压力测试，重点分析 2020 年新冠危机期间 BSM 模型的定价行为，量化静态波动率假设在尾部风险事件中的失效程度。

二、关键代码及运行结果

2.1 BSM 核心定价函数

```
def bsm_call(S, K, T, r, sigma):
    """
    European Call Option Price (BSM)
    S: current stock price
    K: strike price
    T: time to expiration (years)
    r: risk-free rate
    sigma: volatility
    """
    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    call = S*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)
    return call

def bsm_put(S, K, T, r, sigma):
    """
    European Put Option Price (BSM)
    S: current stock price
    K: strike price
    T: time to expiration (years)
    r: risk-free rate
    sigma: volatility
    """
    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    put = K*np.exp(-r*T)*norm.cdf(-d2) - S*norm.cdf(-d1)
    return put
```

```
def chooser_option(S, K, T, tc, r, sigma):
    """
    Chooser Option Price (BSM-based)
    T: final expiration (years)
    tc: choice date (years)
    """
    # Call component: expires at T
    call = bsm_call(S, K, T, r, sigma)

    # Put component: discounted strike, expires at tc
    K_adj = K * np.exp(-r*(T-tc))
    put = bsm_put(S, K_adj, tc, r, sigma)

    # Chooser = Call + Put
    price = call + put
    return price, call, put
```

定义好函数，包括输入值。先通过分别定义 call price 和 put price，然后再相加来定义 chooser_option 的价格

2.2 论文参数配置

```
params = {
    "S": 150, # current stock price (matches paper)
    "K": 150, # strike price
    "T": 1.0, # time to expiration (1 year)
    "tc": 0.5, # choice date (6 months)
    "r": 0.05, # risk-free rate (5%)
    "sigma": 0.20, # volatility (20%)
}
```

参数配置
(论文一致)

```
# Calculate chooser option price
price, call, put = chooser_option(
    S=params["S"],
    K=params["K"],
    T=params["T"],
    tc=params["tc"],
    r=params["r"],
    sigma=params["sigma"]
)
```

使用之前定义的函数，
输入配置好的参数

```
print("=" * 45)
print(" Chooser Option Pricing - BSM Model")
print("=" * 45)
print(f" Stock Price (S) : ${params['S']}")
print(f" Strike Price (K) : ${params['K']}")
print(f" Expiration (T) : {params['T']} year")
print(f" Choice Date (tc) : {params['tc']} year")
print(f" Risk-free Rate (r) : {params['r']*100}%")
print(f" Volatility (sigma) : {params['sigma']*100}%")
print("=" * 45)
print(f" Call Component : ${call:.4f}")
print(f" Put Component : ${put:.4f}")
print(f" Chooser Option Price : ${price:.4f}")
print("=" * 45)
```

论文参数定价运行结果

=====

Chooser Option Pricing - BSM Model

=====

Stock Price (S)	: \$150
Strike Price (K)	: \$150
Expiration (T)	: 1.0 year
Choice Date (tc)	: 0.5 year
Risk-free Rate (r)	: 5.0%
Volatility (sigma)	: 20.0%

→ 输入参数确认

=====

Call Component	: \$15.6759
Put Component	: \$5.1011
Chooser Option Price	: \$20.7770

→ 定价结果

=====

2.3 ATM 定价循环

```
df = pd.read_csv(r"C:\Users\sasa0\Desktop\华泰
\data\featured_dataset.csv", index_col=0, parse_dates=True)
results_atm = []

for date, row in df.iterrows():
    S = row["JPM_Close"]
    K = S # ATM: strike = current price
    r = row["Yield_3M"] / 100
    sigma = row["RealVol_30d"]
    T = 1.0
    tc = 0.5

    if pd.isna(sigma) or sigma <= 0:
        continue

    price, call, put = chooser_option(S, K, T, tc, r, sigma)

    results_atm.append({
        "Date": date,
        "JPM_Close": S,
        "K": K,
        "sigma": sigma,
        "r": r,
        "Call": call,
        "Put": put,
        "Chooser_Price": price,
        "VIX_Regime": row["VIX_Regime"]
    })

results_atm_df = pd.DataFrame(results_atm).set_index("Date")
```

ATM 定价运行结果:

```
print(f"ATM pricing complete: {len(results_atm_df)} trading days")
print(results_atm_df[["JPM_Close", "K", "Call", "Put", "Chooser_Price"]].tail())
```

ATM pricing complete: 1697 trading days

运行成功：共 1697 交易日

Date	JPM_Close	K	Call	Put	Chooser_Price
2024-12-23	231.216553	231.216553	22.042014	7.498525	29.540539
2024-12-24	235.018570	235.018570	22.551939	7.653454	30.205393
2024-12-26	235.823608	235.823608	22.426117	7.627538	30.053656
2024-12-27	233.912872	233.912872	22.325308	7.691293	30.016601
2024-12-30	232.118561	232.118561	22.325075	7.646329	29.971405

关键结果：
选择期权价格

2.4 敏感性分析

```
# Statistics by VIX regime
print("\n-- Chooser Price by VIX Regime --")
regime_stats = results_atm_df.groupby("VIX_Regime")["Chooser_Price"].agg([
    "mean", "std", "min", "max", "count"
]).round(4)
print(regime_stats)

# Call vs Put ratio by regime
print("\n-- Call/Put Ratio by VIX Regime --")
results_atm_df["Call_Put_Ratio"] = results_atm_df["Call"] / results_atm_df["Put"]
ratio_stats = results_atm_df.groupby("VIX_Regime")["Call_Put_Ratio"].mean().round(4)
print(ratio_stats)

# Sigma statistics by regime
print("\n-- Realized Volatility by VIX Regime --")
sigma_stats = results_atm_df.groupby("VIX_Regime")["sigma"].agg([
    "mean", "std", "min", "max"
]).round(4)
print(sigma_stats)
```

计算不同 VIX 制度下的
Call-Put ratio

不同 VIX 下的波
动率统计

制度分类统计运行结果：

-- Chooser Price by VIX Regime --

VIX_Regime	mean	std	min	max	count
High	24.0827	10.3031	11.2604	71.4278	584
Low	17.1891	11.1532	6.1762	70.7005	558
Medium	20.1643	8.6072	6.6146	69.3774	555

-- Call/Put Ratio by VIX Regime --

VIX_Regime	Call_Put_Ratio
High	1.7083
Low	3.1296
Medium	2.2459

高中低 VIX 的
call-put ratio

Name: Call_Put_Ratio, dtype: float64

-- Realized Volatility by VIX Regime --

VIX_Regime	mean	std	min	max
High	0.3554	0.2177	0.1421	1.3506
Low	0.1795	0.0633	0.0935	0.4561
Medium	0.2213	0.0652	0.0879	0.5488

高中低 VIX 的
call-put ratio 的
均值

2.5 BSM 局限性分析

```
# 1. Volatility stability
mean_sigma = results_atm_df["sigma"].mean() * 100
std_sigma = results_atm_df["sigma"].std() * 100
cv_sigma = results_atm_df["sigma"].std() / results_atm_df["sigma"].mean()

# 2. Price gap between regimes
high_mean = results_atm_df.loc[
    results_atm_df["VIX_Regime"] == "High", "Chooser_Price"
].mean()
low_mean = results_atm_df.loc[
    results_atm_df["VIX_Regime"] == "Low", "Chooser_Price"
].mean()
price_gap = high_mean - low_mean
gap_pct = price_gap / low_mean * 100

# 3. Extreme period analysis
covid = results_atm_df.loc["2020-02-01":"2020-05-31"]
normal = results_atm_df.loc["2018-01-01":"2019-12-31"]

covid_mean = covid["Chooser_Price"].mean()
normal_mean = normal["Chooser_Price"].mean()
crisis_ratio = covid_mean / normal_mean
```

BSM 局限性指标运行结果:

```
print("Volatility Stability")
print(f"Mean sigma      : {mean_sigma:.2f}%")
print(f"Std of sigma    : {std_sigma:.2f}%")
print(f"CV (Std/Mean)     : {cv_sigma:.4f}")

print("Price Difference Between VIX Regimes")
print(f"High regime price : ${high_mean:.4f}")
print(f"Low regime price  : ${low_mean:.4f}")
print(f"Price gap         : ${price_gap:.4f} ({gap_pct:.1f}%)")

print("Extreme Period Analysis")
print(f"COVID period price  : ${covid_mean:.4f}")
print(f"Normal period price : ${normal_mean:.4f}")
print(f"Ratio               : {crisis_ratio:.2f}x")
```

```
Volatility Stability
Mean sigma      : 25.37%
Std of sigma    : 15.72%
CV (Std/Mean)   : 0.6197
Price Difference Between VIX Regimes
High regime price : $24.0827
Low regime price  : $17.1891
Price gap         : $6.8935 (40.1%)
Extreme Period Analysis
COVID period price : $36.6603
Normal period price : $12.0802
Ratio             : 3.03x
```

BSM 模型局限性总结：

2.5.1 恒定波动率假设的失效

BSM 模型最核心的假设是波动率 σ 在期权存续期内保持不变。然而实证结果显示，2018 至 2024 年间 JPM 的 30 日实现波动率均值为 25.37%，标准差高达 15.72%，变异系数 $CV=0.62$ 。这意味着真实波动率的波动幅度相当于均值的 62%，与 BSM 假设 $CV=0$ 矛盾。波动率的时变性导致 BSM 在不同市场环境下系统性地高估或低估期权价值，无法适应市场制度的动态切换。

2.5.2 制度依赖性定价差异

基于 IQR 三分位数的 VIX 制度分类结果显示，高波动制度下 Chooser 期权平均价格为 24.08 美元，低波动制度下仅为 17.19 美元，两者相差 40.1%。若使用单一静态 σ 进行定价，则无论市场处于何种制度，模型输出的价格都会趋向同一均值水平，从而在低波动时期系统性高估、在高波动时期系统性低估期权价值。此外，Call/Put 比率在不同制度下也存在显著差异：高波动制度下比率为 1.71，接近 1（双向风险对等）；低波动制度下比率高达 3.13，说明市场在平静时期更倾向于看涨方向。这种制度依赖特征完全无法被 BSM 的静态参数所捕捉。

2.5.3 极端市场表现

2020 年 2 月至 5 月的新冠危机期间，Chooser 期权平均价格达到 36.66 美元，是 2018 至 2019 年正常时期（12.08 美元）的 3.03 倍，这一价格跳升直接源于 VIX 在 2020 年 3 月飙升至历史极值，而 BSM 若采用历史均值波动率进行定价，将严重低估危机时期的期权价值，对持有者造成系统性的风险低估。尾部风险事件的不可预测性和价格非线性跳升，正是 BSM 模型在极端市场中失效的根本原因。

2.6 Github 自动化定价

```
name: Auto BSM Pricing

on:
  workflow_run:
    workflows: ["Auto Data Update"]
    types:
      - completed
  workflow_dispatch:

permissions:
  contents: write

jobs:
  bsm-pricing:
    runs-on: ubuntu-latest
    if: ${{ github.event.workflow_run.conclusion == 'success' || github.event_name == 'workflow_dispatch' }}
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"
      - name: Install dependencies
        run: pip install pandas numpy scipy
```

```

- name: Run BSM pricing
  run: python scripts/bsm_pricing.py
- name: Commit results
  run: |
    git config --global user.email "action@github.com"
    git config --global user.name "GitHub Action"
    git add data/processed/bsm_results.csv
    git commit -m "Auto BSM pricing $(date)" || echo "No changes"
    git push

```

自动收集结果：

← Auto BSM Pricing

✓ Auto BSM Pricing #3

Summary

All jobs

✓ bsm-pricing

Run details

🕒 Usage

📄 Workflow file

Triggered via workflow run 8 hours ago

Status

Total duration

Artifacts

🌐 xzha0487 completed ↗ f65b271

Success

30s

—

bsm_pricing.yml

on: workflow_run

✓ bsm-pricing

27s

3. 可视化图表

3.1 敏感性分析

BSM Chooser Option - Sensitivity Analysis

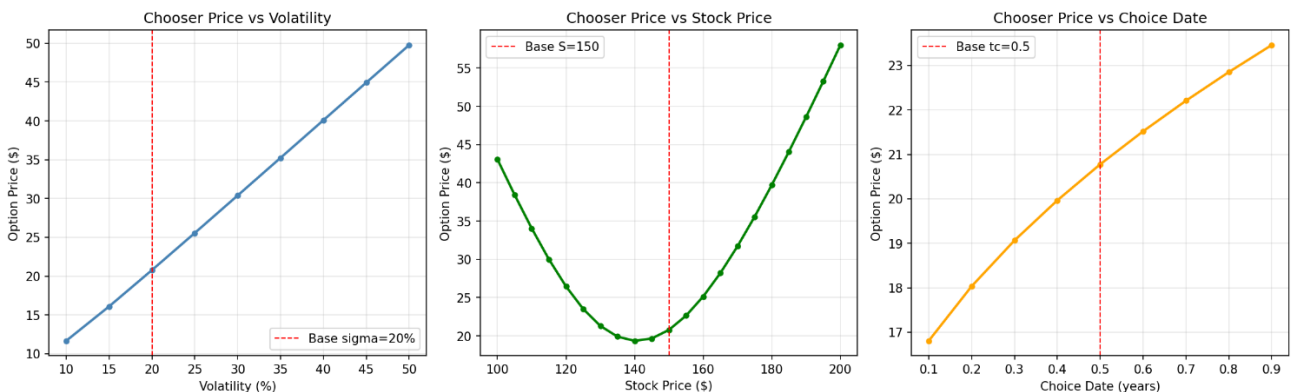


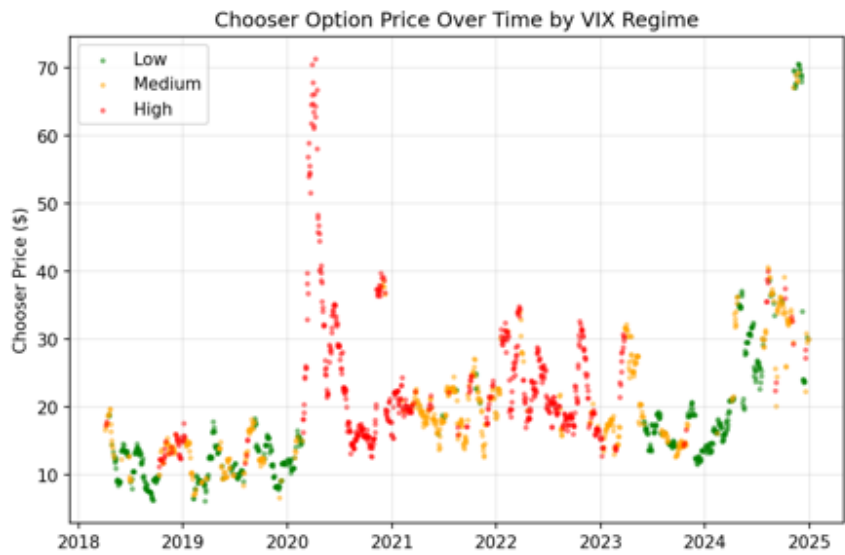
图 1：价格 vs 波动率：线性正相关，波动率越高期权价格越贵，符合预期因为不确定性越大期权价值越高。

图 2：价格 vs 股价：呈 U 型曲线，在 $S=150$ （平值）时价格最低，深度实值或虚值时价格都会上升，这是 Chooser 期权的典型特征。

图 3：价格 vs 选择日：单调递增，选择日越晚持有者拥有更长时间观察市场再做决策，灵活性更高所以价格更贵。

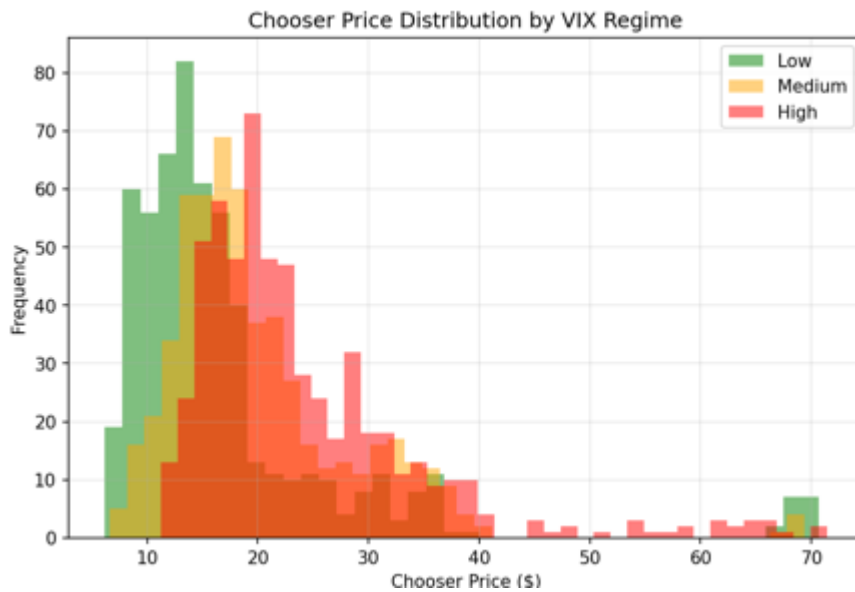
3.2 BSM 性能分析

3.2.1 Chooser 价格时间序列：



2020 年新冠期间出现明显的价格 spike（最高\$71），红色高波动点集中在 2020-2022 年，2018-2019 年和 2023-2024 年价格相对平稳，制度分类颜色分布清晰。

3.2.2 价格分布：



三个制度的分布明显不同：Low 制度集中在\$10-15，High 制度分布更宽且右尾更长，说明高波动时期定价不确定性更大。

3.2.3 波动率 vs 价格：



线性正相关，验证了 BSM 模型中 σ 是定价的核心驱动因素，同时也暴露了 BSM 的局限，现实中这个关系并不是完美线性的。

3.2.4 Call/Put 比率：



2020 年比率急剧下降接近 1（Call 和 Put 几乎等价），说明市场极度恐慌时双向风险对等。2022-2024 年比率异常升高，反映了加息周期中市场持续看涨偏向。

4. 当前局限性

4.1 数据局限性

仅以 JPMorgan Chase (JPM) 作为唯一标的资产，样本覆盖范围有限，模型的泛化能力尚未经过验证。不同行业、不同市值的股票在波动率特征和期权定价行为上存在显著差异，仅基于单一金融股构建的定价模型，在应用于其他标的时可能产生较大偏差。

4.2 模型局限性

当前 BSM 模型基于多个理想化假设，包括波动率恒定、市场无摩擦、股价服从对数正态分布等，这些假设在真实市场中均存在不同程度的违背。尤其是恒定波动率假设，已通过实证分析证明 $CV=0.62$ ，与 BSM 假设 $CV=0$ 存在根本性矛盾。此外，本项目目前使用历史实现波动率作为 σ 的代理，而非市场隐含波动率，两者之间存在一定偏差。

4.3 定价基准局限性

由于无法直接获取 CME 真实 Chooser 期权的历史成交价格，缺乏真实市场价格作为定价基准，无法计算严格意义上的模型定价误差（MAE、RMSE）。当前的性能评估主要基于制度间对比和极端时期分析，而非与真实成交价格的直接对比，这在一定程度上限制了误差分析的客观性。

5. 下周计划

下周将引入机器学习框架，采用两条路线并行推进：第一条是用 LSTM、Random Forest、XGBoost 预测动态波动率后代入 BSM 定价；第二条是直接训练端到端监督学习模型（线性回归、GBDT、神经网络）完成定价。在机器学习框架基础上进行超参数调优（网格搜索/随机搜索），训练最终模型并在测试集上评估，通过 MAE、RMSE、 R^2 与 BSM 基准进行全面对比。同时引入 SHAP/LIME 方法分析各特征对定价结果的贡献程度。